



18BIG

Big Data Tools & Architecture

Lecture 03 — Parquet & the Medallion Architecture

www.bighub.ai

Your guides to applied AI



■ Recap: last time

Three questions from **Session 02 — Data Sources & Bronze Ingestion**:

1. What belongs in **Bronze**, and what do we deliberately **not** do there?
2. What is a **watermark**, and why can't it see **hard deletes**?
3. Why do we design every ingestion job around *"what if this runs twice?"* — what's that property called?

Warm-up — answer out loud before we build on it.

■ Recap: answers

1. **Bronze** keeps data *as it arrived* (raw, duplicates and bad rows included) + lineage metadata. We **don't** clean, deduplicate, reshape or enforce a schema — that's Silver.
2. A **watermark** is the last value loaded (e.g. `max(order_id)`); the next run reads only rows above it. A **hard delete** removes the row, so nothing rises above the watermark → it's invisible (needs a full compare or CDC).
3. **Idempotence** — re-running must not duplicate or corrupt data, so retries, backfills and replays are safe.

■ Today

- **Row vs. columnar** — why analytics loves columns
- **Inside a Parquet file** — row groups, pages, footer
- **Compression & encoding** — why Parquet is small
- **Pushdown** — projection + predicate, skip what you don't need
- **Partitioning** — Hive-style folders & partition pruning
- **Over-partitioning** — the small-files trap
- **File format vs. table format** — the road to Delta (S4)
- **Hands-on** — a measurement lab: *prove* pruning helps



Part 1

Row vs. columnar storage

Your guides to applied AI



■ The question behind the format

Analytics asks **narrow, deep** questions: *"total revenue last month"* touches **2 columns** across **millions of rows** — not whole rows.

Row storage (OLTP)

- Row stays together on disk
- Great to **fetch / update one record**
- A column scan reads **every byte** of every row

Columnar storage (OLAP)

- Each column stored **contiguously**
- Read **only the columns you select**
- Similar values sit together → **compress hard**

Parquet is columnar — built for the scans a lakehouse runs all day

■ Why columns compress better

Same column, same kind of data

- A `country` column is a handful of repeated strings
- An `order_ts` column is near-sorted numbers
- Neighbours are **similar** → tiny after encoding

Encodings do the work

- **Dictionary** — store each distinct value once, rows become small ints
- **RLE** — "CZ × 4 000" instead of 4 000 copies
- Then **Snappy / Zstd** on top

Row storage mixes an int, a string and a timestamp on every row — far less for a compressor to exploit.



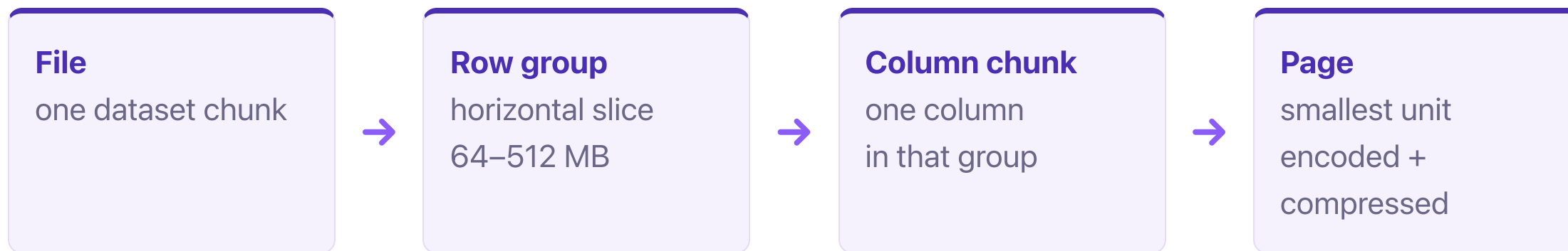
Part 2

Inside a Parquet file

Your guides to applied AI



■ The anatomy



- A **footer** at the end holds the schema and **per-row-group statistics** (min / max / null counts).
- A **PAX** layout: row groups split across workers; columns inside each group give pruning + compression.

Inspired by Google's Dremel paper; the open Apache Parquet format since 2013.

■ The footer is the magic

Before reading any data, an engine reads the footer and decides **what it can skip**.

What's in it

- Schema (columns, types)
- Row-group locations
- **Min / max / null count** per column, per row group

What it enables

- **Projection pushdown** — jump straight to the columns you select
- **Predicate pushdown** — skip whole row groups that can't match
- Parallelism — hand each row group to a worker

- **Row-group size is a tuning knob**

Too small	Too large
metadata overhead explodes	fewer groups to skip (coarse pruning)
worse compression	less parallelism
many tiny pages	high memory per read

64–512 MB per row group is the production sweet spot — smaller for latency-sensitive lakes, larger for archival.



Part 3

Pushdown & partitioning

Your guides to applied AI



■ Two kinds of pushdown

Projection pushdown

- `SELECT amount, order_ts ...`
- Engine reads **only those column chunks**
- The other columns are **never touched** on disk

Predicate pushdown

- `WHERE amount > 35`
- Row group's **max is 20** → skip the whole group
- Driven by the **footer statistics**

Both happen inside a single file — before partitioning even enters the picture

■ Partitioning: pushdown at the folder level

Hive-style partitioning splits the dataset into **folders named** `col=value/` :

```
orders_by_date/  
  order_date=2026-09-28/  data.parquet  
  order_date=2026-09-29/  data.parquet  
  order_date=2026-09-30/  data.parquet    ← a query for the 30th reads only this
```

- `WHERE order_date = DATE '2026-09-30'` → the engine **lists folders** and opens just one.
- **Partition pruning** skips entire folders *before* opening a single file; predicate pushdown then skips row groups *inside* it.

Two layers of skipping stack: prune folders, then prune row groups.

■ Choose the partition key well

Good keys

- **Low cardinality**, matches how you **filter**
- Date binned to **day / month** (not raw timestamp)
- Region, category, source system

The over-partitioning trap

- High-cardinality key (raw `order_ts`, `customer_id`) → **thousands of tiny files**
- Many small files run **slower** than a few big ones
- Fix: **bin to coarser granularity**

Aim for files around **128–512 MB**. Partition to *prune*, not to shard into dust.



Part 4

File format vs. table format

Your guides to applied AI



■ Parquet is just files

What Parquet gives you

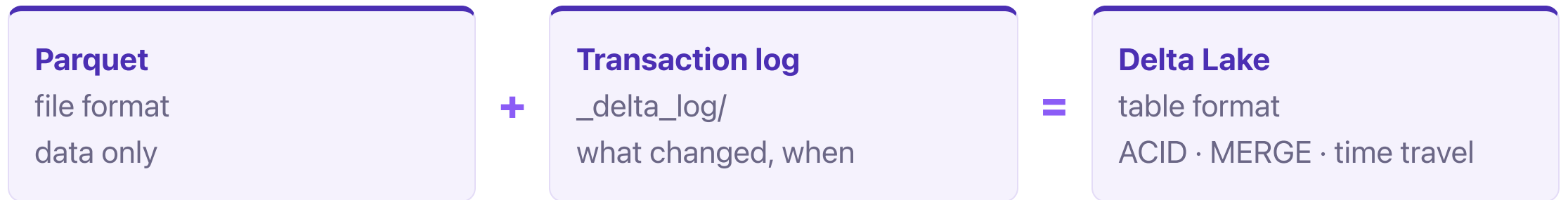
- Columnar, compressed, self-describing
- Pushdown + statistics
- Read by everything (DuckDB, Polars, Spark...)

What it does **not** give you

- No **transactions** (ACID)
- No reliable **upsert / MERGE**
- No **history / time travel**
- No safe **concurrent writes**

A folder of Parquet files is a snapshot, not a table. Who's reading while you overwrite?

■ The road to Delta



- A **table format** wraps Parquet with a log → transactions, upserts, versions.
- Delta / Iceberg / Hudi all do this; same idea, different implementations.
- **Session 04:** we build the first Silver **Delta** table with version history.



Hands-on

Prove that pruning helps

Your guides to applied AI



■ Run it

```
git pull && uv sync
uv run eshop datagen && uv run eshop ingest    # if you don't have Bronze yet
uv run python sessions/03-parquet-medallion/exercise/partition_lab.py
```

Then look at the physical layout partitioning creates:

```
ls data/lab/orders_by_date/    # one order_date=YYYY-MM-DD/ folder per day
```

On 20k orders the milliseconds are noisy — the point is *why* pruning helps, not the exact number.

■ Today's exercise

Fill in the `# TODO` s in `sessions/03-parquet-medallion/exercise/partition_lab.py` :

1. Write Bronze orders as a **Hive-partitioned** dataset (one folder per `order_date`).
2. Pick a day to filter on (`df["order_date"].max()`).
3. Complete the pruned query's **WHERE** so it filters on the **partition column**.
4. Write the data to **CSV** and compare **CSV vs. Parquet** file size.
5. **(harder)** Build **two-level** `year/month` partitioning and run a pruned month query.

■ The key lines

```
# Task 1 – partitioned write (Polars)
df.write_parquet(part_dir, partition_by=["order_date"])

# Task 3 – pruned read (DuckDB): filter on the partition column
con.execute(
    f"SELECT count(*) FROM read_parquet('{part}', hive_partitioning=true) "
    f"WHERE order_date = DATE '{target_day}'"
)

# Task 5 – two-level partitioning
df2.write_parquet(ym_dir, partition_by=["order_year", "order_month"])
```

Full scan filters inside one file; the pruned query skips whole folders first. CSV vs. Parquet shows the compression ratio.

■ Key takeaways

- **Columnar** storage reads only the columns you select and compresses them hard — built for analytics.
- A Parquet file nests **row groups → column chunks → pages**; the **footer** carries the statistics.
- **Projection + predicate pushdown** skip columns and row groups *before* reading data.
- **Partitioning** = pushdown at the folder level; prune to filter, but beware **over-partitioning / small files**.
- Parquet is a **file format** — no ACID. A **table format** (Delta) adds a log → next session.

■ Next time

Session 04 — Delta Lake: a table format on top of Parquet

- Why a Data Lake alone isn't enough: ACID, reliable upserts, history
- **MERGE** upsert and **time travel** over older versions
- The first **Silver Delta** table in our pipeline

■ References & further reading

- Columnar storage formats — Parquet, ORC, Arrow (ClickHouse) — clickhouse.com
- Parquet file anatomy: row groups, column chunks, pages, metadata — dev.to/databro
- Partition pruning (MotherDuck glossary) — motherduck.com
- Pros and cons of Hive-style partitioning — delta.io/blog
- Delta Lake (ACID table format) — delta.io

The background of the slide features a series of concentric, wavy lines in various shades of purple, creating a sense of depth and movement.

bighub

Thank you

Let's build.

www.bighub.ai

Your guides to applied AI

